InsideInsight

Phase 2 — Pricing Analytics & Feature Engineering

Handoff Guide · market_trend_v2.ipynb

1. Overview

This document is the official handoff guide for the Phase 2 Analytics notebook (market_trend_v2.ipynb). It describes every analytical step performed, the design decisions made, key outputs, and the schema of the gold_features Delta table that Phase 4 (AI Agent) and Phase 5 (Dashboard) consume directly.

1-1. Data Dictionary — gold_features

Table path: workspace.default.gold_features (Delta Lake, Hive Metastore).

Column Name	Type	Description	Business Logic
listing_id	LONG	Unique Airbnb listing identifier	Primary key. Sourced from gold_listings.
city	STRING	City where the listing is located	Used for regional market segmentation.
neighborhood	STRING	Cleansed neighborhood name	Primary unit for price benchmarking.
room_type	STRING	Property type (e.g., Entire home)	Required for apples-to-apples comparison.
price	DOUBLE	Current nightly rate (USD)	Baseline for all gap analysis.
median_neighborhood_price	DOUBLE	Median price for same room type in neighborhood	The 'Fair Market Value' reference.
price_gap_pct	DOUBLE	% deviation from neighborhood median	Negative = underpriced; Positive = overpriced.
occupancy_rate	DOUBLE	Rolling 90-day occupancy rate (0–1)	Derived from gold_calendar_daily.is_available.
occupancy_rank_percentile	DOUBLE	Occupancy rank within neighborhood (0–100)	0 = lowest, 100 = highest booker in area.
amenity_score	DOUBLE	Relative amenity score (0–100)	Normalized vs. max in same neighbourhood+room.

Column Name	Type	Description	Business Logic
review_rank_percentile	DOUBLE	Review count rank within neighborhood (0–100)	Proxy for guest trust and market demand.
competitive_score	DOUBLE	Composite 0–100 competitiveness score	30% price + 25% amenity + 25% review + 20% occ.
peak_month	INTEGER	Month number with highest occupancy (1–12)	0 if occupancy data unavailable (coalesced).
peak_occupancy_rate	DOUBLE	Avg occupancy rate during peak month	Rounded to 4 decimal places.

2. Source Tables

All four Gold Tables written by Phase 1 are consumed. Load them at the top of the notebook:

```
gold_listings       = spark.table('workspace.default.gold_listings')
gold_reviews        = spark.table('workspace.default.gold_reviews')
gold_calendar       = spark.table('workspace.default.gold_calendar')
gold_calendar_daily = spark.table('workspace.default.gold_calendar_daily')

# Verify row counts
print('listings      :', gold_listings.count())
print('calendar      :', gold_calendar.count())
print('calendar_daily:', gold_calendar_daily.count())
print('reviews       :', gold_reviews.count())
```

Table	Primary Key	Used For	Notes
gold_listings	listing_id	Pricing, scoring, ML features	price column is valid here; calendar price is null
gold_calendar	listing_id + date	Reference only	Pricing columns are null city-wide — do not use for price analysis
gold_calendar_daily	listing_id + calendar_date	Occupancy modeling (Steps 2–3)	is_available: 0 = booked, 1 = available
gold_reviews	listing_id	Not used in v2 directly	Available for future sentiment enrichment

⚠ gold_calendar price columns are null across all cities. All price analysis must use gold_listings.price.

3. Step-by-Step Walkthrough

STEP 1 Pricing Distribution Analysis

Goal: Establish per-city, per-neighborhood, per-room-type price benchmarks and clean outliers before all downstream calculations.

Step 1-A — IQR Outlier Detection

v2 uses a city-level IQR upper bound instead of the global 3-sigma method used in earlier drafts. This handles cities like Seattle that contain extreme luxury listings ($50,000+) which would otherwise inflate every neighborhood median.

Algorithm:

- Compute Q1 and Q3 per city using percentile_approx.
- IQR = Q3 − Q1
- city_upper_bound = Q3 + 1.5 × IQR
- Left-join thresholds back to gold_listings.

```
city_stats = gold_listings.groupBy('city').agg(
    F.percentile_approx('price', 0.25).alias('q1'),
    F.percentile_approx('price', 0.75).alias('q3')
)
city_stats = city_stats \
    .withColumn('iqr', F.col('q3') - F.col('q1')) \
    .withColumn('city_upper_bound', F.col('q3') + 1.5 * F.col('iqr'))

gold_listings_with_bounds = gold_listings.join(
    city_stats.select('city', 'city_upper_bound'),
    on='city', how='left'
)
```

Step 1-B — Data Cleaning & Visual Verification

Listings above their city's upper bound are filtered out to produce clean_listings. A box plot is rendered to visually confirm that extreme values have been removed before any scoring logic runs.

```
clean_listings = gold_listings_with_bounds.filter(
    F.col('price') <= F.col('city_upper_bound')
```

```
)

# Visual verification
pdf_clean = clean_listings.select('city', 'price').toPandas()
sns.boxplot(x='city', y='price', hue='city', data=pdf_clean, palette='Set2')
plt.title('Price Distribution by City (After IQR Filtering)')
plt.show()
```

STEP 2 Occupancy Modeling

Goal: Derive demand intensity for every listing from gold_calendar_daily and identify peak/off-peak months per neighborhood. All occupancy logic uses gold_calendar_daily (not gold_calendar).

⚠ Column name: the date field in gold_calendar_daily is calendar_date. The availability field is is_available (integer 0/1). Primary key is listing_id.

Step 2-A — Overall & 90-Day Rolling Occupancy

Two occupancy figures are produced:

- occupancy_rate — overall lifetime occupancy for cross-validation against Phase 1's availability_rate.
- occupancy_rate_90d — rolling 90-day window for recent demand trend detection. This is the value used in all downstream scoring.

```
# Overall occupancy (cross-validation only)
occupancy_raw = gold_calendar_daily.groupBy('listing_id').agg(
    F.count('*').alias('total_days'),
    F.sum(F.when(F.col('is_available') == 0, 1).otherwise(0)).alias('booked_days')
)
occupancy_raw = occupancy_raw.withColumn(
    'occupancy_rate',
    (F.col('booked_days') / F.col('total_days')).cast('double')
)

# 90-day rolling window (primary signal for AI Agent)
w90 = Window.partitionBy('listing_id') \
            .orderBy('calendar_date') \
            .rowsBetween(-89, 0)

occ_90d_df = gold_calendar_daily \
    .withColumn('is_booked', F.when(F.col('is_available') == 0, 1).otherwise(0)) \
    .withColumn('booked_90d', F.sum(F.col('is_booked')).over(w90)) \
```

```
    .withColumn('occupancy_rate_90d', F.col('booked_90d') / 90)
```

Step 2-B — Peak Month Identification

Monthly average occupancy_rate_90d is aggregated by city + neighbourhood_cleansed. The month with the highest average becomes peak_month for that neighborhood, published into peak_months and later joined into gold_features.

```
# city already exists in occ_90d_df — only pull neighbourhood from listings
monthly_occ = occ_90d_df \
    .join(clean_listings.select('listing_id', 'neighbourhood_cleansed'),
          on='listing_id', how='inner') \
    .groupBy('city', 'neighbourhood_cleansed',
             F.month('calendar_date').alias('month')) \
    .agg(F.avg('occupancy_rate_90d').alias('avg_monthly_occ'))

w_peak = Window.partitionBy('city', 'neighbourhood_cleansed') \
               .orderBy(F.col('avg_monthly_occ').desc())

peak_months = monthly_occ \
    .withColumn('rank', F.rank().over(w_peak)) \
    .filter(F.col('rank') == 1) \
    .select('city', 'neighbourhood_cleansed',
            F.col('month').alias('peak_month'),
            F.round('avg_monthly_occ', 4).alias('peak_occupancy_rate'))
```

STEP 3 Competitiveness Score

Goal: Compute a single 0–100 composite score capturing each listing's relative market position within its neighborhood.

Feature	Calculation	Weight	Window Partition
price_gap_pct	((price − median) / median) × 100	30%	neighbourhood_cleansed + room_type
amenity_score	amenities_count / neigh_max × 100, capped at 100	25%	neighbourhood_cleansed + room_type
review_count_rank	percent_rank() on number_of_reviews	25%	neighbourhood_cleansed
occupancy_rank	percent_rank() on occupancy_rate_90d	20%	neighbourhood_cleansed

Step 3-A — price_gap_pct

Window is partitioned by neighbourhood_cleansed AND room_type to ensure apples-to-apples comparisons (e.g., private rooms are never benchmarked against entire homes).

```
window_neighborhood = Window.partitionBy('neighbourhood_cleansed', 'room_type')

listings_scored = clean_listings \
    .withColumn('median_neighborhood_price',
                F.percentile_approx('price', 0.50).over(window_neighborhood)) \
    .withColumn('price_gap_pct',
                ((F.col('price') - F.col('median_neighborhood_price')) /
                 F.col('median_neighborhood_price') * 100).cast('double'))
```

Interpretation:

- Negative value (e.g., −15%) → listing is priced below market → competitive advantage.
- Positive value (e.g., +25%) → listing is overpriced → AI Agent will flag for investigation.

Step 3-B — amenity_score

Normalizes amenities_count relative to the maximum in the same neighbourhood + room_type group. Score is capped at 100 to prevent outlier mega-listings from compressing the rest of the distribution.

```
window_context = Window.partitionBy('neighbourhood_cleansed', 'room_type')

listings_scored = listings_scored \
    .withColumn('neigh_avg_amenities',
F.avg('amenities_count').over(window_context)) \
    .withColumn('neigh_max_amenities',
F.max('amenities_count').over(window_context)) \
    .withColumn('amenity_score',
                F.least(F.lit(100.0),
                        (F.col('amenities_count') / F.col('neigh_max_amenities')
                         * 100).cast('double')))
```

Step 3-C — Percentile Rankings (review_count_rank & occupancy_rank)

Critical ordering: the join that adds occupancy_rate_90d must happen BEFORE the occupancy window is defined. Defining the window on a column that does not yet exist causes a silent null-fill.

```
window_review =
Window.partitionBy('neighbourhood_cleansed').orderBy('number_of_reviews')

# JOIN FIRST — occupancy_rate_90d must exist before window definition
listings_scored = listings_scored \
    .join(occ_90d_df.select('listing_id', 'occupancy_rate_90d').distinct(),
          on='listing_id', how='left')

window_occupancy =
Window.partitionBy('neighbourhood_cleansed').orderBy('occupancy_rate_90d')

listings_scored = listings_scored \
    .withColumn('review_count_rank', F.percent_rank().over(window_review)) \
    .withColumn('occupancy_rank',    F.percent_rank().over(window_occupancy))

# Fill nulls for brand-new listings (no booking history)
listings_scored = listings_scored.fillna({
    'review_count_rank': 0,
    'occupancy_rank':    0,
    'occupancy_rate_90d': 0,
})
```

Step 3-D — Weighted competitive_score

All four features are normalized to a 0–100 scale and combined. Note the price_gap_norm direction: a listing priced below the median gets a higher score (50 − price_gap_pct, not 50 + price_gap_pct).

```
# price_gap_norm: negative gap → competitively priced → higher score
listings_scored = listings_scored.withColumn(
    'price_gap_norm',
    F.greatest(F.lit(0.0), F.least(F.lit(100.0), 50 - F.col('price_gap_pct')))
)

listings_scored = listings_scored.withColumn(
    'competitive_score',
    (
        F.col('price_gap_norm')     * 0.30 +
        F.col('amenity_score')      * 0.25 +
        F.col('review_count_rank') * 100 * 0.25 +
        F.col('occupancy_rank')     * 100 * 0.20
    ).cast('double')
)
```

AI Agent thresholds:

- Score > 80 → Market Leader: well-optimized, may support further price increases.
- Score 40–80 → Mid-tier: targeted improvements available.
- Score < 40 → Fixer-Upper: immediate intervention required (price cut or amenity upgrade).

STEP 4 gold_features Table — Final Delivery

Goal: Publish a single, clean, AI-ready Delta table at workspace.default.gold_features.

Step 4-A — Join Peak Season Data

```
feature_df = listings_scored.join(
    peak_months.select('city', 'neighbourhood_cleansed',
                       'peak_month', 'peak_occupancy_rate'),
    on=['city', 'neighbourhood_cleansed'],
    how='left'
)
```

Step 4-B — Column Selection & Formatting

Percentile ranks (0–1) are multiplied by 100 and exposed as occupancy_rank_percentile and review_rank_percentile for direct human readability. Numerical fields are rounded.

```
gold_features = feature_df.select(
    F.col('listing_id'),
    F.col('city'),
    F.col('neighbourhood_cleansed').alias('neighborhood'),
    F.col('room_type'),
    F.round('price', 2).alias('price'),
    F.round('median_neighborhood_price', 2).alias('median_neighborhood_price'),
    F.round('price_gap_pct', 2).alias('price_gap_pct'),
    F.round('occupancy_rate_90d', 4).alias('occupancy_rate'),
    F.round(F.col('occupancy_rank') * 100, 2).alias('occupancy_rank_percentile'),
    F.round('amenity_score', 2).alias('amenity_score'),
    F.round(F.col('review_count_rank') * 100, 2).alias('review_rank_percentile'),
    F.round('competitive_score', 2).alias('competitive_score'),
    F.coalesce(F.col('peak_month'), F.lit(0)).alias('peak_month'),
    F.round('peak_occupancy_rate', 4).alias('peak_occupancy_rate')
)
```

Step 4-C — Write to Delta Table

overwriteSchema=true is required because the schema evolves between notebook runs (especially after Step 5 adds predicted_optimal_price).

```
gold_features.write \
    .format('delta') \
    .mode('overwrite') \
    .option('overwriteSchema', 'true') \
    .saveAsTable('workspace.default.gold_features')

print(f'Step 4 Complete. Records: {gold_features.count()}')
display(spark.table('workspace.default.gold_features'))
```

Step 4-D — Spot-Check Validation

A random sample of 10 rows is printed in a structured format for manual sanity checking. Verify that price_gap_pct, occupancy_rate, and competitive_score are directionally consistent before signing off.

```
sample = spark.table('workspace.default.gold_features') \
              .sample(fraction=0.01).limit(10).toPandas()

for _, row in sample.iterrows():
    print(f"Listing {row['listing_id']} | {row['city']} | {row['neighborhood']}")
    print(f"  Room Type      : {row['room_type']}")
    print(f"  Price          : ${row['price']:.2f}")
    print(f"  Neigh Median   : ${row['median_neighborhood_price']:.2f}")
    print(f"  Price Gap      : {row['price_gap_pct']:.2f}%")
    print(f"  Occupancy      : {row['occupancy_rate']:.1%}")
    print(f"  Occ Rank %ile  : {row['occupancy_rank_percentile']:.1f}")
    print(f"  Amenity Score  : {row['amenity_score']:.1f}")
    print(f"  Comp. Score    : {row['competitive_score']:.1f}")
    print(f"  Peak Month     : Month {int(row['peak_month'])}")
    print('-' * 50)
```

STEP 5 ML Pricing Model (GBTRegressor) — Experimental, could be ingnored

📋 Status: Experimental. Best R^2 achieved = 0.4775 (below the 0.6 acceptance threshold). RMSE ≈ 60.91. **The predicted_optimal_price column was NOT added to gold_features in this run.**

Step 5-A — Feature Engineering

```
feature_cols = [
    'amenities_count', 'availability_rate', 'number_of_reviews',
    'review_scores_rating', 'accommodates', 'bedrooms', 'beds'
]
cat_cols = ['room_type', 'city']

indexers = [StringIndexer(inputCol=c, outputCol=c+'_idx', handleInvalid='keep')
            for c in cat_cols]
assembler = VectorAssembler(
    inputCols=feature_cols + [c+'_idx' for c in cat_cols],
    outputCol='features', handleInvalid='skip'
)
```

Step 5-B — Training & Evaluation

```
train_df, test_df = clean_listings.randomSplit([0.8, 0.2], seed=42)

gbt      = GBTRegressor(labelCol='price', featuresCol='features', maxIter=50)
pipeline = Pipeline(stages=indexers + [assembler, gbt])
model    = pipeline.fit(train_df)

rmse = RegressionEvaluator(metricName='rmse').evaluate(model.transform(test_df))
r2   = RegressionEvaluator(metricName='r2'  ).evaluate(model.transform(test_df))
print(f'RMSE: {rmse:.2f}   R²: {r2:.4f}')
```

Step 5-C — Conditional Write-Back

```
if r2 > 0.6:
    all_preds = model.transform(clean_listings) \
        .select('listing_id', F.col('prediction').alias('predicted_optimal_price'))

    spark.table('workspace.default.gold_features') \
        .join(all_preds, on='listing_id', how='left') \
        .write.format('delta').mode('overwrite') \
        .option('overwriteSchema', 'true') \
        .saveAsTable('workspace.default.gold_features')

    print(f'predicted_optimal_price added. R²={r2:.4f}')
else:
    print(f'Model R² ({r2:.4f}) < 0.6 — column skipped.')
```

Feature importance finding: Price Gap and Location scores (city index) were the highest contributors to predicted price. Bedrooms and accommodates followed closely. This suggests the model captures location

premium well but struggles with within-neighborhood variance, likely due to unlisted quality signals (photos, descriptions).

5. Deliverables

Deliverable	Format	Location / Destination	Status
gold_features Delta table	Delta Lake	workspace.default.gold_features	✅ Complete
market_trend_v2.ipynb	Databricks Notebook	GitHub repo /notebooks/phase2/	✅ Complete
Data Dictionary (this doc §4)	Word / PDF	GitHub repo /docs/	✅ This document
Pricing box plot visualization	PNG (notebook output)	Phase 5 Dashboard assets	⚠ Export manually
ML model metrics (RMSE / R^2)	Notebook cell output	README — Phase 5 section	📋 R^2=0.48, RMSE=60.91
predicted_optimal_price column	DOUBLE (Delta column)	gold_features (conditional)	⏭ Skipped — $R^2 < 0.6$

6. Known Issues for Phase 4

Known Issues

- ML model (Step 5) did not reach the $R^2 > 0.6$ threshold. Best run: R^2=0.4775, RMSE=60.91. Predicted price column is absent from gold_features.
- peak_month defaults to 0 (not null) for listings with no occupancy history — Phase 4 prompts should handle month=0 as 'unknown season'.
- occupancy_rate_90d is 0 for new listings with no calendar data — review_count_rank and occupancy_rank are also set to 0 via fillna.